Your Planning Poker® Game Summary

CO7095 CodeGauge - Backlog Estimation

Estimating all 36 user stories for CodeGauge, our CO7095 group project. Fibonacci deck, max 3 rounds per story, reference story US01 = 3 points. Screenshot every round - the assignment requires evidence of planning poker.

Story	Story Title	Score
1	US01 [S1][Gunjan] classify_line() - As a reviewer I want each source line classified as blank, comment or code so that the tool can count the three kinds separately	3
2	US02 [S1][Gunjan] count_leading_spaces() - As a reviewer I want a line's indentation measured in spaces so that nesting depth can be derived from it	5
3	US03 [S2][Gunjan] depth_from_indent() - As a reviewer I want an indentation width converted to a nesting depth so that deeply nested code can be flagged	5
4	US04 [S2][Gunjan] comment_ratio_percent() - As a reviewer I want comment density as a percentage so that poorly documented files can be flagged	5
5	US05 [S3][Harshita] comment_ratio_band() - As a reviewer I want comment density banded so that the report can describe it in words	5
6	US06 [S3][Gunjan] line_length_grade() - As a reviewer I want each line's length graded against a limit so that over-long lines can be reported by seriousness	8
7	US07 [S1][Gunjan] is_function_header() - As a reviewer I want function definition lines recognised so that the tool knows where each function starts	5
8	US08 [S2][Gunjan] count_parameters() - As a reviewer I want the parameters in a definition counted so that functions with too many parameters can be flagged	8
9	US09 [S3][Gunjan] scan_file() - As a reviewer I want a source file reduced to a record per function so that every later stage works on numbers instead of text	13
10	US10 [S1][Kasam] decision_points() - As a reviewer I want the decision keywords in a function totalled so that cyclomatic complexity can be computed from them	5
11	US11 [S1][Saaket] cyclomatic_complexity() - As a reviewer I want each function's cyclomatic complexity computed so that over-complex functions can be flagged	8
12	US12 [S1][Kasam] max_nesting_depth() - As a reviewer I want the deepest nesting level in a function found so that deeply nested code can be flagged	8
13	US13 [S2][Kasam] nesting_penalty() - As a reviewer I want penalty points charged for excess nesting so that the score reflects how far past the limit the code is	8
14	US14 [S2][Kasam] parameter_count_rating() - As a reviewer I want a parameter count rated so that the report can describe it in words	5
15	US15 [S3][Kasam] function_length_rating() - As a reviewer I want a function's length rated so that over-long functions can be described in words	5
16	US16 [S3][Kasam] maintainability_index() - As a reviewer I want a maintainability index per function so that complexity and documentation are	13

	weighed together	
17	US17 [S2][Kasam] duplicate_block_score() - As a reviewer I want duplication expressed as a percentage so that copied code can be flagged	13
18	US18 [S3][Kasam] complexity_rating() - As a reviewer I want complexity, nesting and parameters combined into one rating so that the report has a single per-function verdict	13
19	US19 [S1][Saaket] rule_complexity() - As a reviewer I want functions over the complexity limit reported so that I know which functions to simplify	5
20	US20 [S1][Kasam] rule_nesting() - As a reviewer I want functions nested too deeply reported so that I know which functions to flatten	5
21	US21 [S2][Saaket] rule_length() - As a reviewer I want over-long functions reported so that I know which functions to split	8
22	US22 [S2][Saaket] rule_params() - As a reviewer I want functions with too many parameters reported so that I know which signatures to simplify	5
23	US23 [S2][Saaket] rule_comment_ratio() - As a reviewer I want insufficiently commented files reported so that I know which files need documenting	8
24	US24 [S3][Saaket] rule_line_length() - As a reviewer I want over-long lines reported so that I know which lines to wrap	5
25	US25 [S3][Saaket] rule_duplication() - As a reviewer I want duplicated blocks reported so that I know which code to factor out	13
26	US26 [S1][Saaket] classify_severity() - As a reviewer I want a measurement banded against its limit so that every rule grades its findings on one consistent scale	5
27	US27 [S3][Saaket] combine_severities() - As a reviewer I want a list of severities reduced to the worst one so that a function can be summarised by its most serious finding	5
28	US28 [S1][Harshita] weighted_penalty() - As a reviewer I want each severity to carry a penalty weight so that serious findings cost more than trivial ones	8
29	US29 [S1][Harshita] score_from_issues() - As a reviewer I want a quality score from the issues found so that a file's state is a single number	13
30	US30 [S1][Harshita] grade_band() - As a reviewer I want a letter grade from the score so that the result is readable at a glance	5
31	US31 [S2][Harshita] compare_scores() - As a reviewer I want two reviews compared so that I can see whether a file improved or regressed	8
32	US32 [S3][Harshita] trend_direction() - As a reviewer I want the trend across my review history described so that I can see the direction of travel	5
33	US33 [S2][Harshita] serialise_report() - As a reviewer I want a report turned into text lines so that it can be stored without a database	8
34	US34 [S2][Harshita] parse_record_line() - As a reviewer I want stored lines parsed back and corrupt ones rejected so that one bad line does not destroy my history	13
35	US35 [S3][Harshita] history_statistics() - As a reviewer I want the minimum, maximum and mean of my scores so that I can summarise a file's history	5
36	US36 [S3][Gunjan] save_review and load_history() - As a reviewer I want reviews appended to a file and read back so that my review history survives restarting the tool	21